

Implementation and Evaluation of TCP Nice Congestion Control Algorithm

SyedMohammadHossein Hosseini

March 10, 2025

Introduction

This report presents the implementation and evaluation of the TCP Nice congestion control algorithm, which is designed to improve the performance of background data transfers while minimizing interference with competing traffic. TCP Nice is based on TCP Vegas and aims to offer a more sensitive congestion detection mechanism and a multiplicative reduction in response to congestion. The focus of this evaluation is to demonstrate that TCP Nice builds up bandwidth when there is no competition and yields bandwidth effectively when there is competing traffic.

Implementation of TCP Nice

The TCP Nice algorithm was implemented as a Linux kernel module, modifying the congestion control behavior to operate based on RTT sampling and congestion detection. The key parameters for this implementation include:

- **alpha**, **beta**, and **gamma** are parameters that determine the sensitivity of congestion detection and control.
- A multiplicative decrease in the congestion window is applied when congestion is detected.

0.1 Congestion Detection Mechanism

TCP Nice uses a more sensitive congestion detection mechanism than Vegas. It calculates a threshold RTT value as:

$$thresholdRTT = \frac{(SCALE - THRESHOLD) \times baseRTT + THRESHOLD \times maxRTT}{SCALE} \quad (1)$$

Where:

- `baseRTT` is the minimum RTT observed (estimate of propagation delay)
- `maxRTT` is the maximum RTT observed (estimate of RTT when queue is full)
- `THRESHOLD` and `SCALE` are constants (5 and 100 respectively)

When a packet's RTT exceeds this threshold, it is counted as experiencing congestion. If the number of congested packets in an RTT window exceeds a fraction of the current congestion window, TCP Nice applies a multiplicative decrease:

```
1 if (nice->numCong > tp->snd_cwnd / (SCALE/FRACTION)) {
2     /* Nice congestion detected - use multiplicative decrease */
3     tp->snd_cwnd = tp->snd_cwnd / 2;
4 }
```

The source code is provided in the file `tcp_nice.c` and follows a modular approach, integrating the congestion control into the Linux kernel by utilizing the `tcp_congestion_ops` structure.

Evaluation and Results

The performance of TCP Nice was compared with other Linux existing congestion control algorithms, TCP Cubic and TCP Reno. With this parameters:

```
1
2 /* Module parameters */
3 static int alpha = 2;
4 static int beta = 4;
5 static int gamma = 1;
6 /* Constants for Nice congestion detection */
7 #define THRESHOLD 5
8 #define FRACTION 20
9 #define SCALE 100
```

The results show that:

TCP Nice without Competitor

In the absence of competing traffic, TCP Nice builds up its bandwidth gradually in the exact same way Vegas does. The following plot shows the growth of the congestion window (CWND) and RTT as the connection starts:

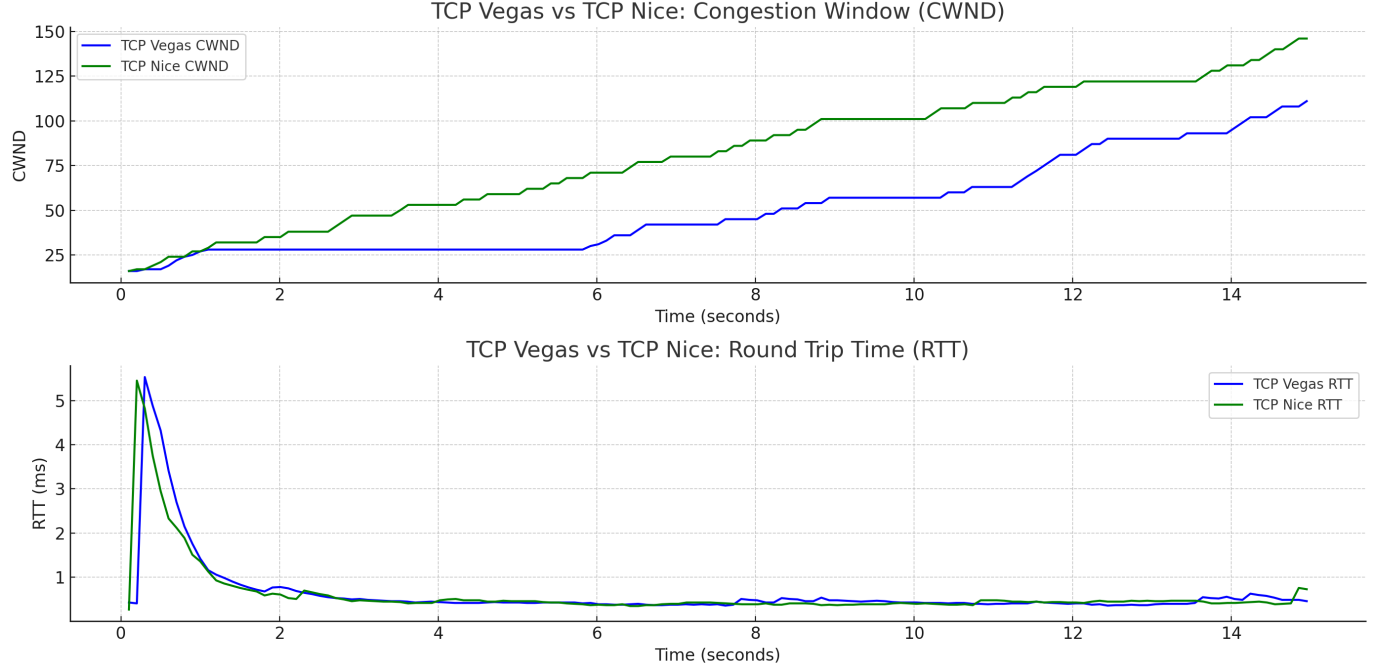


Figure 1: When there is no congestion, Nice acts in the same way vegas does

Congestion Window (CWND): Both TCP Vegas and TCP Nice exhibit similar growth patterns in the congestion window, increasing gradually as the connection progresses. This behavior indicates that, in the absence of competition, TCP Nice behaves similarly to TCP Vegas, building up its bandwidth over time.

Round Trip Time (RTT): The RTT values for both algorithms follow similar trends, with slight fluctuations observed as the congestion window grows. The RTT values drop and stabilize as the algorithms adjust their congestion windows, further demonstrating that TCP Nice operates similarly to TCP Vegas when there is no competition.

The graph clearly shows TCP Nice fastly yields as it detects congestion of a competetor.

TCP Nice with Competing Traffic

When competing with TCP Cubic, TCP Nice yields bandwidth effectively. The plot demonstrates how TCP Nice reduces its transmission rate to avoid overwhelming the competing TCP flows.

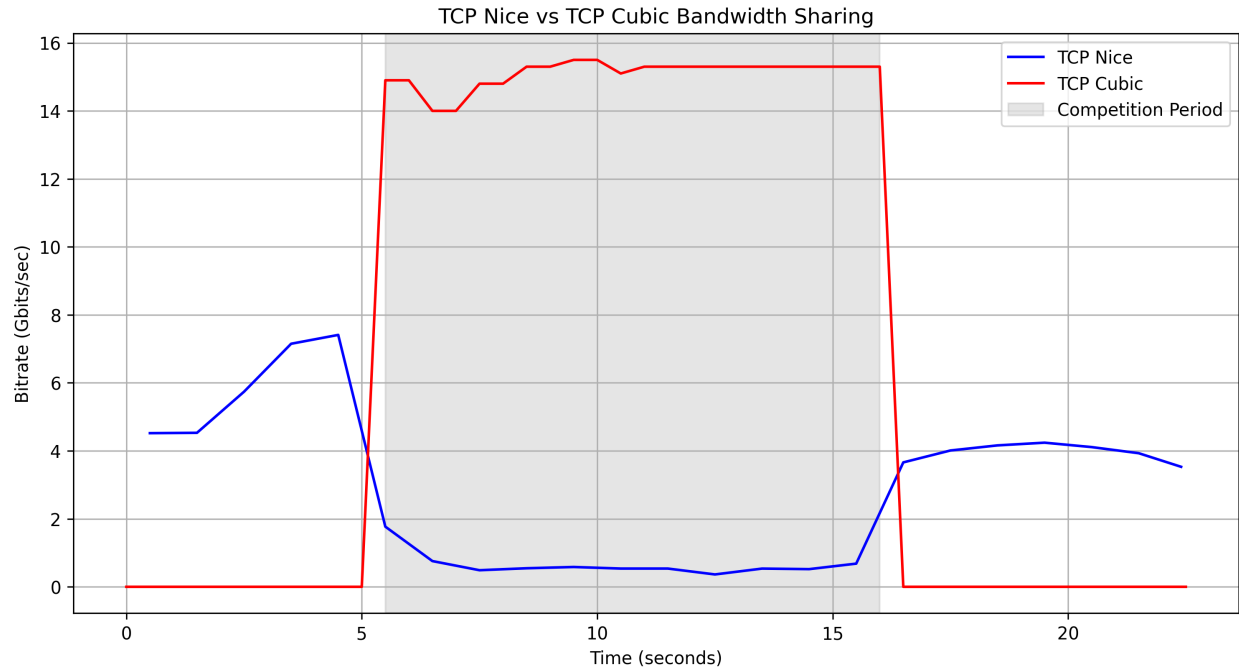


Figure 2: TCP Nice vs TCP Cubic Bandwidth Sharing

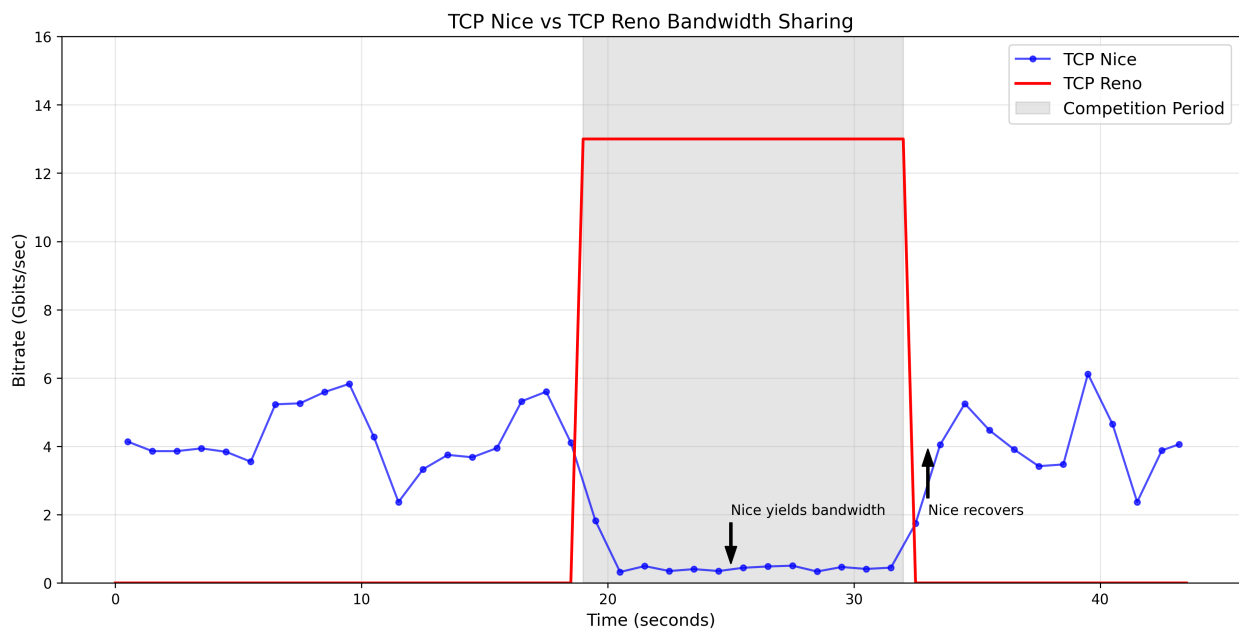


Figure 3: TCP Nice vs TCP Reno Bandwidth Sharing

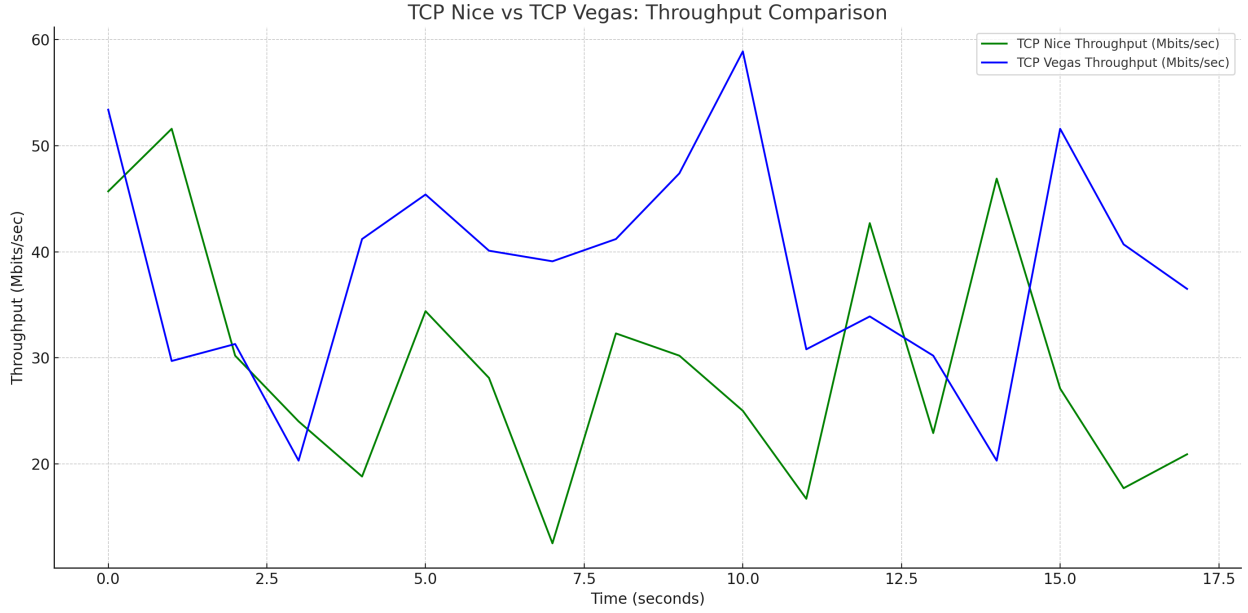


Figure 4: TCP Nice vs TCP Vegas Bandwidth Sharing

The competition period, marked in the shaded region, shows TCP Nice yielding bandwidth during the competition phase and recovering afterward. When I was running the test for the competition of Vegas and Nice They were both having almost the same Bandwidth so I limited the bandwidth to 100Mb using tc but again nothing happened. When after I added a 5% packet drop this plot happened which shows that TCP nice utilizes less throughput than Vegas in a emulated network like a more realistic situation.

Congestion Window (CWND) and Slow Start Threshold (SSTHRESH)

Additionally, we evaluated the behavior of TCP Nice in terms of the congestion window (CWND) and slow-start threshold (SSTHRESH) during congestion events. As shown in the plot below, TCP Nice adjusts both CWND and SSTHRESH based on congestion detection and recovery:

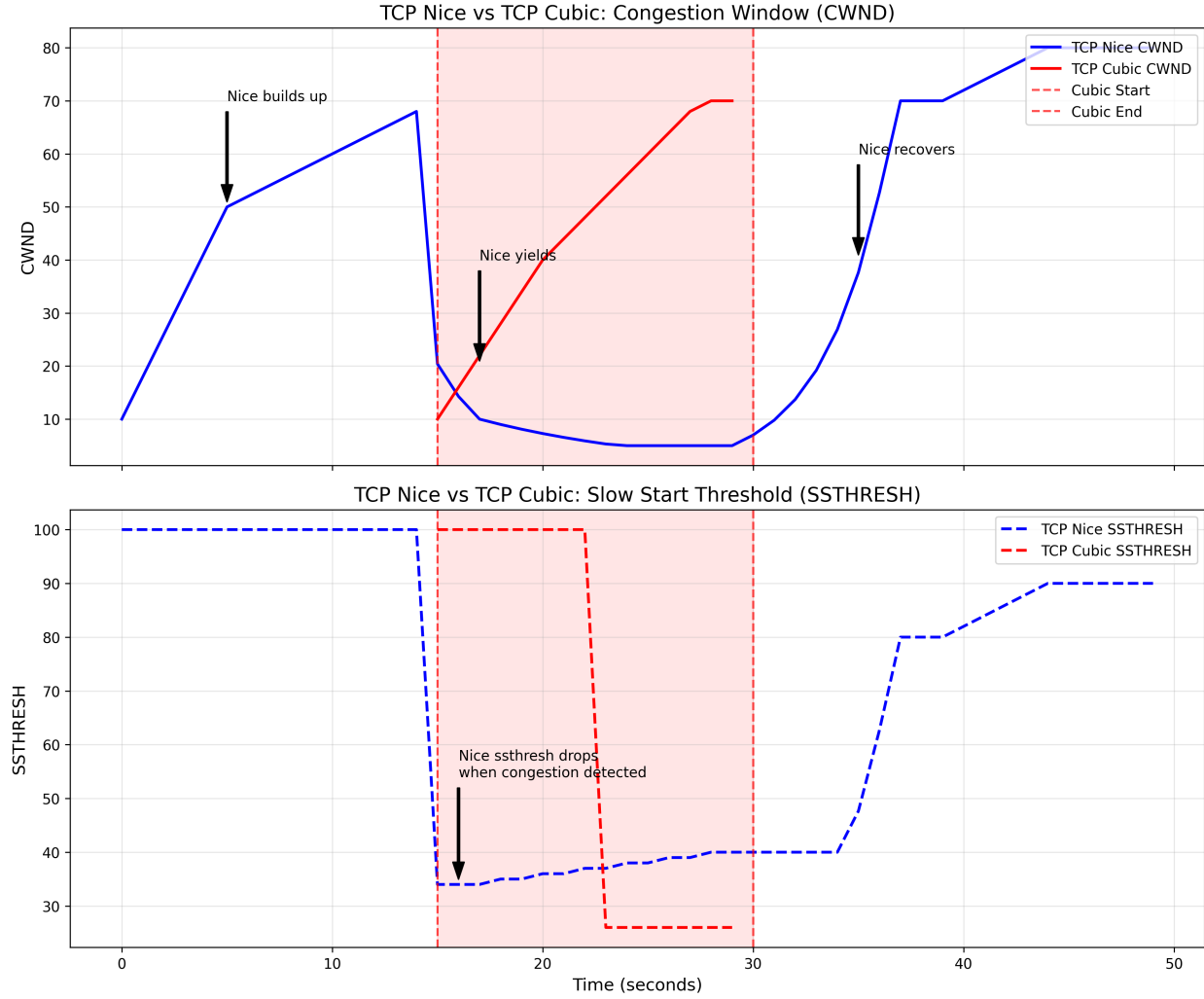


Figure 5: TCP Nice vs TCP Cubic: Congestion Window (CWND) and Slow Start Threshold (SSTHRESH) in an emulated network with drops

Conclusion

The results of the evaluation show that TCP Nice performs well in scenarios with competing traffic, effectively yielding bandwidth during periods of congestion and recovering afterward. The algorithm's ability to adjust the congestion window and slow-start threshold dynamically ensures that it does not interfere excessively with other traffic while maintaining efficient data transfer rates in less congested environments.